



## GIN CHEAT SHEET

Core setup, initialization, operations &amp; integration quick reference

PART 1: SETUP &amp; CORE

## 01. SETUP &amp; ROUTER

Installation

```
go get github.com/gin-gonic/gin
r := gin.Default() // includes Logger+Recovery...
r.Run(":8080")      // listen and serve
r := gin.New()     // bare router (no middle...
gin.SetMode(gin.ReleaseMode) // production
```

## 04. MIDDLEWARE

Workflow

```
// Global:
r.Use(gin.Logger(), gin.Recovery())
r.Use(CorsMiddleware())
// Group-level:
api := r.Group("/api", AuthMiddleware())
// Custom:
func AuthMiddleware() gin.HandlerFunc {
return func(c *gin.Context) {
if !isValid(c) { c.AbortWithStatus(401); retu...
c.Next()
}
}
```

## 02. ROUTES &amp; HTTP METHODS

Architecture

```
r.GET("/users", listUsers)
r.POST("/users", createUser)
r.PUT("/users/:id", updateUser)
r.DELETE("/users/:id", deleteUser)
r.PATCH r.HEAD r.OPTIONS r.Any
r.NoRoute(func(c *gin.Context){ c.JSON(404, g...
```

## 05. REQUEST BINDING

CRUD API

```
// Bind JSON body:
var req CreateUserReq
if err := c.ShouldBindJSON(&req); err != nil {
c.JSON(400, gin.H{"error": err.Error()})
return
}
// Bind query params:
c.ShouldBindQuery(&params)
// Bind form:
c.ShouldBind(&form)
// With validation tags: `binding:"required,e...
```

## 03. ROUTE PARAMS &amp; QUERY

YAML / Config

```
r.GET("/users/:id", func(c *gin.Context) {
id := c.Param("id")
page := c.DefaultQuery("page", "1")
search := c.Query("q")
c.JSON(200, gin.H{"id": id, "page": page})
})
c.QueryArray("ids") // repeated ?ids=1&ids=2
```

## 06. RESPONSE RENDERING

Integration

```
c.JSON(200, gin.H{"users": users})
c.JSON(http.StatusCreated, newUser)
c.String(200, "Hello %s", name)
c.XML(200, xmlData)
c.File("./static/index.html")
c.Stream(200, func(w io.Writer) bool { ... })
c.Status(204) // no body
```



# SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

## 07. GROUPING ROUTES

Hardening

```
v1 := r.Group("/v1")
{
  v1.GET("/users", ...) // /v1/users
  v1.POST("/users", ...)
  admin := v1.Group("/admin", AdminOnly())
  admin.GET("/stats", ...) // /v1/admin/stats
}
```

## 10. COMMON CONTEXT METHODS

Unit Tests

```
c.Set("user", user) // set value in con...
c.Get("user") // retrieve (return...
c.MustGet("user") // panics if not fo...
c.Request // *http.Request
c.Writer // http.ResponseWr...
c.ClientIP() // client IP
c.GetHeader("Authorization")
```

## 08. ERROR HANDLING

Observability

```
c.Error(err) // attach error to context
c.AbortWithStatusJSON(401, gin.H{"error": "un...
c.AbortWithStatus(403)
// Global error handler:
r.Use(func(c *gin.Context) {
  c.Next()
  for _, e := range c.Errors { log.Error(e) }
})
```

## 11. COMMON GOTCHAS

Commands

c.JSON() after c.Abort() still writes always return after Abort  
Bind() calls c.Abort on failure; ShouldBind() doesn't  
gin.H is just map[string]any  
ReleaseMode silences debug logs enable in production  
Goroutines in handlers: copy context values before starting

## 09. VALIDATION

Optimization

```
type CreateUser struct {
  Name string `json:"name" binding:"required,m...
  Email string `json:"email" binding:"required,...
  Age int `json:"age" binding:"gte=0,lte=1...
}
// Custom validator:
v, _ := binding.Validator.Engine().(*validato...
v.RegisterValidation("isbn", validateISBN)
```